

Apresentação T1 – MST

Dupla: Gabriel Rangel Lustosa; Davi Lima de Oliveira Rocha

Algoritmo utilizado: Kruskal (com comparação prática com o Prim)

1. Modelagem do problema como grafo:

O que o problema pede? Na cidade descrita pelo problema (Dark roads), o prefeito procura economizar no custo de energia desligando as luzes de certas ruas. Contudo, ele precisa garantir que haja pelo menos uma rota iluminada para o pedestre caminhar entre quaisquer pontos, mantendo a cidade totalmente conectada enquanto gasta o mínimo possível. Isso resume perfeitamente o conceito de uma Árvore Geradora Mínima (MST), onde buscamos manter um grafo conectado removendo as arestas redundantes e mantendo apenas as de menor custo.

Como o problema pede a quantidade máxima de dinheiro que pode ser economizada, a resposta é obtida calculando a diferença entre o custo de todas as estradas originais e o peso total da MST encontrada.

2. Estratégia algorítmica derivada da modelagem:

O que o Kruskal pede? O algoritmo de Kruskal adota uma estratégia gulosa (ganancioso) baseada em arestas. Ele pede que eu sempre escolha a menor aresta disponível no momento que não crie um ciclo no grafo. Por isso, ele percorrerá as conexões, analisará quais têm os menores pesos, verificará se os vértices de cada aresta já estão conectados (para evitar ciclos) e garantirá a seleção ótima.

Seu funcionamento:

- Ordena todas as arestas do grafo em ordem crescente de peso.
- Começa com um grafo vazio (onde cada vértice é um componente isolado).
- Vai adicionando as arestas uma a uma:
 - Se NÃO forma ciclo → adiciona a aresta à MST e une os componentes (usando Union-Find).
 - Se forma ciclo → ignora a aresta e passa para a próxima.

O contraponto com o PRIM:

Para fins de estudo e validação, também foi implementado o algoritmo de Prim para ver como ele se comportaria. No Prim, o fluxo foca em “como a árvore deve crescer” radialmente a partir de um vértice inicial. Em vez da checagem global de ciclos que o Kruskal faz, o Prim realiza o controle verificando se o vértice vizinho já foi ou não visitado, garantindo que não haja repetições.

Funcionamento do Prim:

- Começa a partir de um vértice inicial.
- Marca este vértice como visitado.
- Coloca as arestas vizinhas em uma fila de prioridade indexada.
- Escolhe a menor aresta que leva a um novo vértice não visitado e repete o processo.

Suas complexidades:

Dentro do Kruskal, a complexidade de tempo é de $O(E \log E)$, isso porque ele precisa ordenar todas as E arestas do grafo no início. No Prim, a complexidade é de $O(E \log V)$, onde cada inserção/remoção na fila de prioridade indexada custa $\log V$, repetindo-se para as arestas. Fazendo a comparação teórica para este problema esparsos onde o número de arestas é próximo ao de vértices ($E \approx V$), o tempo de execução calculado no papel deveria ser muito próximo.

Por que o Kruskal é o nosso ator principal?

Neste trabalho, a escolha e o uso do algoritmo de Kruskal se mostram os mais adequados para a solução principal, pois ele opera diretamente sobre o conjunto de arestas. Isso está perfeitamente alinhado com a forma como os dados de entrada são fornecidos no problema Dark roads (uma lista direta de conexões (x, y, z), tornando a nossa implementação muito mais direta, limpa e intuitiva.

O algoritmo de Prim, embora eficiente, exige uma mudança na estrutura de recepção dos dados para listas de adjacência e um controle muito mais complexo sobre a expansão da árvore através do gerenciamento de uma fila de prioridade indexada (IndexMinPQ), tornando a engenharia do código menos direta em comparação à simplicidade elegante do Kruskal com Union-Find.

3. Complexidade e casos especiais:

Apesar de Kruskal e Prim resolverem o mesmo problema e terem complexidades teóricas semelhantes, existem cenários práticos onde um se destaca mais que o outro.

O Kruskal se destaca fortemente em grafos esparsos e quando a entrada do problema já é nativamente uma lista de arestas, como ocorre no Dark roads. Nesses casos, ele aproveita melhor a estrutura natural dos dados sem exigir mapeamentos extras de ponteiros.

O Caso Especial (Análise Prática no Juiz Online):

Como um diferencial do nosso trabalho, submetemos ambas as implementações no Online Judge para analisar o comportamento real dos algoritmos sob a ótica da Java Virtual Machine (JVM). O resultado trouxe um caso especial de desempenho:

- Prim: 1.550 segundos (Accepted)
- Kruskal: 1.870 segundos (Accepted)

Essa diferença de 0.320 segundos ocorre porque, em Java, o Kruskal precisa gerenciar todas as 200.000 arestas dentro de uma fila de prioridade global, gerando uma árvore de Heap muito alta e maior alocação de objetos. Já o Prim, por limitar o tamanho da sua fila ao número de vértices (**V**), trabalha com uma estrutura mais enxuta em memória, o que poupa o Garbage Collector do Java.

4. Conclusão

Portanto, a escolha de um algoritmo não depende apenas da sua complexidade matemática, mas principalmente da arquitetura do problema e da linguagem utilizada. No caso do Dark roads, o Kruskal consolidou-se como a nossa abordagem principal por ser a solução mais direta, intuitiva e compatível com o formato da entrada, enquanto o Prim funcionou como um excelente caso especial de estudo sobre otimização de estruturas de dados e gerenciamento de memória em baixo nível.

Resultado de cada:

My Submissions

#	Problem	Verdict	Language	Run Time	Submission Date
31146237	11631 Dark roads PRIM	Accepted	JAVA	1.550	2026-05-24 18:40:37
31145907	11631 Dark roads KRUSKAL	Accepted	JAVA	1.870	2026-05-24 15:07:32

Imagem 1: Captura de tela de Online Judge